

C# Fundamentals

Class 1 — Syntax, Structure, and How C# Works Under the Hood

What is C#?

Modern & Object-Oriented

C# is a modern, object-oriented language developed by Microsoft for building robust applications

.NET Runtime

Runs on the .NET runtime - write once, run anywhere with .NET's cross-platform capabilities

Universal Applications

Used for web, desktop, mobile, games, and cloud applications across all platforms

Basic Program Structure

- Every C# app starts with a **Main method** - the entry point
- Code is organized into **namespaces, classes, and methods**
- Statements end with a **semicolon ;**
- Curly braces **{ }** define blocks of code

Code Example

```
using System;

namespace HelloWorld
{
    class Program
    {
        static void Main()
        {
            Console.WriteLine("Hello World");
        }
    }
}
```

Data Types in C#

Value Types

Store data directly

- int, float, double
- char, bool, decimal

Reference Types

Store a reference to the data

- string, object
- Arrays, classes

C# is statically typed - you must declare the type of every variable

Variables and Constants

Variable

A named container for data. Value can change.

Constant

Value is fixed at compile time. Declared with `const`.

Code Example

```
// Variables
int score = 100;
score = 150; // OK

// Constants
const double Pi = 3.1416;
// Pi = 3.14; // ERROR! Cannot change a constant
```

Naming Rules

- Must start with a letter or underscore
- Cannot use reserved keywords (e.g., class, int)
- Use `camelCase` for variables
- Use `PascalCase` for methods and classes

Type Conversion

1

Implicit Conversion

Automatic, safe (small type → large type)

2

Explicit Conversion (Casting)

Manual, requires care (large type → small type)

3

Convert Class

Use `Convert` class or `.Parse()` / `.ToString()` for non-compatible types

Code Example

```
// Implicit
int num = 10;
double bigNum = num; // OK: int fits into double

// Explicit (Casting)
double price = 9.99;
int rounded = (int)price; // Result: 9 (decimal truncated)

// Using Convert
string input = "123";
int result = Convert.ToInt32(input); // Result: 123
```

Operators and Expressions

Arithmetic

+, -, *, /, %

Assignment

=, +=, -=, *=, /=

Comparison

==, !=, <, >, <=, >=

Logical

&& (AND), || (OR), ! (NOT)

Code Example

```
int a = 10;
int b = 3;
Console.WriteLine(a + b); // 13
Console.WriteLine(a / b); // 3 (integer division)
Console.WriteLine(a % b); // 1 (remainder)
bool isValid = (a > 5) && (b < 5); // true
```

Control Flow – If / Else

- Programs need to make decisions
- `if` checks a condition. `else` handles the alternative.
- Use `else if` for multiple conditions.

Code Example

```
int age = 20;

if (age < 13)
{
    Console.WriteLine("Child");
}
else if (age < 20)
{
    Console.WriteLine("Teenager");
}
else
{
    Console.WriteLine("Adult");
}
```


Control Flow – Switch and Loops

Switch

Cleaner than multiple if statements for single values

For Loop

Best when you know the number of iterations

While Loop

Best when the condition is dynamic

Code Example

```
// Switch
int day = 3;
switch (day) {
    case 1:
        Console.WriteLine("Wednesday");
        break;
    case 2:
        Console.WriteLine("Thursday");
        break;
    default:
        Console.WriteLine("Other day");
        break;
}
```

```
// For loop
for (int i = 0; i < 5; i++) {
    Console.WriteLine(i);
}
```

```
// While Loop
int i = 0;
while (i < 5)
{
    Console.WriteLine(i);
    i++;
}
```

Arrays

- An array stores multiple values of the same type in a fixed-size container
- Index starts at 0
- Size is set at creation and cannot change

Code Example

```
// Declaration and initialization
int[] numbers = new int[3];
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;

// Shortcut
string[] names = { "Ali", "Sara", "John" };
Console.WriteLine(names[1]); // Output: Sara

// names[3] = "Tom"; // ERROR: Index out of bounds!
```

Collections – List

- `List` is a dynamic array - it grows and shrinks automatically
- Part of `System.Collections.Generic`
- Much more flexible than arrays for real-world apps

Code Example

```
using System.Collections.Generic;

List<string> fruits = new List<string>();
fruits.Add("Apple");
fruits.Add("Banana");
fruits.Remove("Apple");
Console.WriteLine(fruits.Count); // Output: 1

// Shortcut initialization
List<int> scores = new List<int> { 90, 85, 88 };
```

Methods

- A method is a block of code that performs a specific task
- Helps avoid repetition and organize code
- Has a return type, name, and parameters

Code Example

```
// Method with return value
static int Add(int a, int b)
{
    return a + b;
}

// Method with no return value (void)
static void Greet(string name)
{
    Console.WriteLine("Hello, " + name);
}

// Calling methods
int sum = Add(5, 3); // sum = 8
Greet("Ali"); // Output: Hello, Ali
```

Procedural vs OOP Paradigm

Procedural Programming

Code runs top-to-bottom, uses functions and variables

- Good for small scripts
- Hard to maintain as code grows

Object-Oriented Programming (OOP)

Code is organized into objects

- Objects combine data (fields) and behavior (methods)
- Easier to reuse, maintain, and scale

① **Analogy:** Procedural is like a recipe list (step 1, step 2, step 3). OOP is like a kitchen with tools (Oven, Mixer, Fridge) that have their own functions.

Access Modifiers



public

Accessible from anywhere



private

Accessible only inside the class



protected

Accessible in class and derived classes



internal

Accessible within the same project/assembly



Best Practice: Make fields private, expose them via public properties or methods.

Value Types vs Reference Types

Value Types

Store the actual data. Stored on the Stack.

- int, double, bool, char
- struct

Reference Types

Store a memory address pointing to data. Stored on the Heap.

- string, class, object
- array

Key Difference

- Assigning a value type = creates a copy
- Assigning a reference type = creates a shared reference

Code Example

```
// Value Type
int a = 10;
int b = a; // b gets a COPY of 10
b = 20;
Console.WriteLine(a); // Output: 10 (unchanged)

// Reference Type
int[] arr1 = { 1, 2, 3 };
int[] arr2 = arr1; // arr2 points to SAME data
arr2[0] = 99;
Console.WriteLine(arr1[0]); // Output: 99 (changed!)
```

Memory Allocation – Stack vs Heap

Stack

Fast, automatic memory. Stores local variables and method calls.

- Data is removed automatically when the method ends (LIFO)

Heap

Larger, slower memory. Stores objects that need to live longer.

- Managed manually (in C++) or by the Garbage Collector (in C#)

📌 **Analogy:** Stack is like a stack of plates. Add/remove from the top automatically. Heap is like a big warehouse. You request space, and a cleaner removes unused items later.

Where Does Data Live?

Code Example

```
void Calculate()
{
    int x = 10; // Stack
    bool flag = true; // Stack

    Person p = new Person(); // 'p' reference is on Stack
    // Actual Person object is on Heap
}

// When Calculate() ends:
// x, flag, and reference 'p' are removed from Stack instantly
// The Person object on Heap stays until Garbage Collection
```


Constructors

- A constructor is a special method that runs when an object is created
- It has the same name as the class and no return type

Types of Constructors



Default

No parameters. Sets default values.



Parameterized

Accepts arguments to initialize fields.



Copy

Creates a new object from an existing object.

Constructor Examples

Code Example

```
class Car
{
    public string Brand;
    public int Year;

    // Default Constructor
    public Car()
    {
        Brand = "Unknown";
        Year = 0;
    }

    // Parameterized Constructor
    public Car(string brand, int year)
    {
        Brand = brand;
        Year = year;
    }
}

Car c1 = new Car(); // Uses default
Car c2 = new Car("Toyota", 2023); // Uses parameterized
```

Copy Constructor

Code Example

```
class Employee
{
    public string Name;
    public int ID;

    public Employee(string name, int id)
    {
        Name = name;
        ID = id;
    }

    // Copy Constructor
    public Employee(Employee other)
    {
        Name = other.Name;
        ID = other.ID;
    }
}

Employee e1 = new Employee("Ali", 101);
Employee e2 = new Employee(e1); // Copy of e1
```

Garbage Collection (GC)

- C# uses **Automatic Memory Management**
- The **Garbage Collector (GC)** cleans up unused objects from the Heap
- You do not manually delete objects like in C++
- GC runs automatically when memory is low or when the system is idle

Benefits

- Prevents memory leaks
- Prevents dangling pointers

Object Lifetime

- An object is "alive" as long as there is a reference to it
- When no variable points to an object, it becomes eligible for GC
- You can suggest GC to run, but you cannot force it